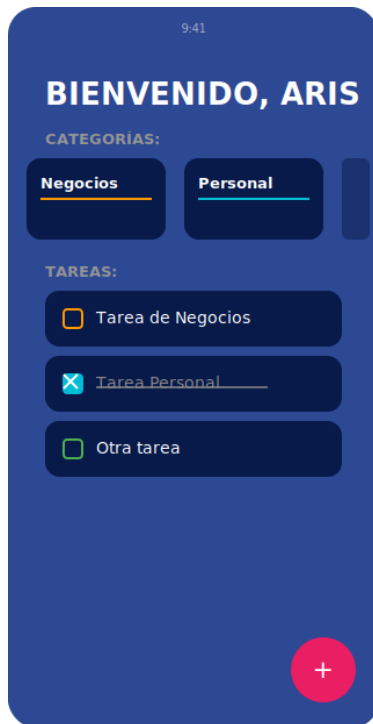


## UNIDAD DIDÁCTICA 4

ToDoApp – Lista de tareas con RecyclerView y Dialogs



Pantalla principal



Dialog añadir tarea

|                           |                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------|
| <b>Módulo</b>             | Programación en Dispositivos Móviles                                                |
| <b>Proyecto</b>           | ToDoApp – Lista de tareas con categorías                                            |
| <b>Nuevos componentes</b> | RecyclerView · Adapter · ViewHolder · Dialog · RadioGroup · CheckBox                |
| <b>Nuevos conceptos</b>   | Sealed class · Funciones lambda · Data class · Estilos XML · Filter/lista funcional |
| <b>Prerrequisitos</b>     | UD1 – Kotlin Fundamentos   UD3 – IMC App (CardView, FAB, lateinit, ContextCompat)   |

## ÍNDICE DE CONTENIDOS

(Generado automáticamente por Word: Referencias > Tabla de contenido)

## PARTE I — MODELO DE DATOS Y CONFIGURACIÓN

## 1. Descripción de la aplicación

### 📌 Qué vamos a construir

Una aplicación de lista de tareas (To-Do) con las siguientes funcionalidades:

- Mostrar tareas agrupadas por categoría (Negocios, Personal, Otros).
- Filtrar tareas según las categorías activas (habilitadas/deshabilitadas).
- Marcar tareas como completadas (el texto se tacha con una línea).
- Añadir nuevas tareas mediante un diálogo flotante.

Esta app introduce el RecyclerView, el componente más importante del desarrollo Android para mostrar listas de forma eficiente.

El aspecto final de la app:

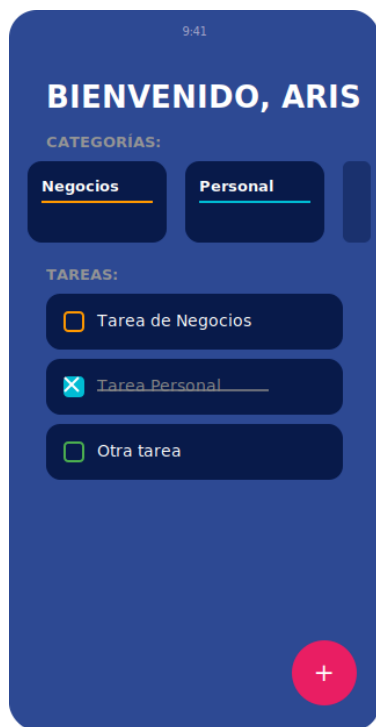


Fig. 1 – Pantalla principal con lista de tareas



Fig. 2 – Diálogo para añadir tarea

## 2. Ficheros del proyecto

| Fichero                 | Tipo         | Función                                            |
|-------------------------|--------------|----------------------------------------------------|
| TaskCategory.kt         | Sealed class | Define las tres categorías posibles                |
| Task.kt                 | Data class   | Modelo de datos de una tarea                       |
| TodoActivity.kt         | Activity     | Lógica principal de la pantalla                    |
| activity_todo.xml       | Layout       | Estructura visual de la pantalla principal         |
| item_task_category.xml  | Layout       | Celda del RecyclerView horizontal (categorías)     |
| item_todo_task.xml      | Layout       | Celda del RecyclerView vertical (tareas)           |
| dialog_task.xml         | Layout       | Vista del diálogo de nueva tarea                   |
| CategoriesAdapter.kt    | Adapter      | Conecta la lista de categorías con el RecyclerView |
| CategoriesViewHolder.kt | ViewHolder   | Pinta cada celda de categoría                      |

|                    |            |                                                |
|--------------------|------------|------------------------------------------------|
| TasksAdapter.kt    | Adapter    | Conecta la lista de tareas con el RecyclerView |
| TasksViewHolder.kt | ViewHolder | Pinta cada celda de tarea                      |

### 3. Crear el proyecto

#### PASO 1 Crear el proyecto Android

File > New > New Project > Empty Views Activity

Name:        ToDoApp

Package name: com.dam.todoapp

Language:    Kotlin

Minimum SDK: API 26 (Android 8.0)

Pulsa Finish y espera a que Gradle sincronice.

### 4. Configurar colors.xml y strings.xml

#### PASO 2 Configurar colors.xml

Abre res > values > colors.xml y añade la paleta ToDoApp.

```
<!-- res/values/colors.xml - paleta ToDoApp -->
<resources>
    <color name="white">#FFFFFF</color>
    <color name="black">#FF000000</color>
    <!-- — Fondos — -->
    <color name="todo_background_todo_app">#FF2D4993</color>    <!-- Fondo principal:
azul -->
    <color name="todo_background_card">#FF081A4A</color>        <!-- Tarjeta oscura --
>
    <color name="todo_background_disabled">#787878</color>      <!-- Categoría
deshabilitada -->
    <!-- — Textos — -->
    <color name="todo_subtitle_text">#959595</color>
    <color name="todo_accent">#E91E63</color>                  <!-- Rosa para el FAB
-->
    <!-- — Colores de categoría — -->
    <color name="todo_business_category">#FF9800</color>        <!-- Naranja: Negocios
-->
    <color name="todo_personal_category">#00BCD4</color>        <!-- Cian: Personal --
>
    <color name="todo_other_category">#4CAF50</color>          <!-- Verde: Otros -->
</resources>
```

#### PASO 3 Configurar strings.xml

Abre res > values > strings.xml y añade los textos de la app.

```
<!-- res/values/strings.xml -->
<resources>
    <string name="app_name">ToDoApp</string>
    <string name="todo_title">Bienvenido, Aris</string>
    <string name="todo_categories">Categorías:</string>
    <string name="todo_tasks">Tareas:</string>
    <string name="todo_dialog_add_task">Añade la tarea</string>
    <string name="todo_dialog_category_business">Negocios</string>
```

```
<string name="todo_dialog_category_personal">Personal</string>
<string name="todo_dialog_category_other">Otros</string>
<string name="todo_dialog_add_task_button">Crear tarea</string>
</resources>
```

- ✓ Después de los Pasos 1-3 tenemos:
- ✓ Proyecto creado y sincronizado.
- ✓ colors.xml con 8 colores (fondos, texto, categorías).
- ✓ strings.xml con 9 cadenas de texto.

## 5. Sealed class: TaskCategory

Una sealed class es una clase cuyos subtipos están restringidos al mismo fichero. El compilador conoce en tiempo de compilación todos los tipos posibles, lo que permite usar when sin cláusula else.

```
// Cualquier TaskCategory solo puede ser uno de estos tres tipos.
// No existe la posibilidad de crear un cuarto tipo desde fuera.
sealed class TaskCategory {
    object Business : TaskCategory()
    object Personal : TaskCategory()
    object Other : TaskCategory()
}
```

La ventaja principal es la exhaustividad garantizada del when:

```
// El compilador garantiza que when es exhaustivo:
when (taskCategory) {
    TaskCategory.Business -> { /* Negocios */ }
    TaskCategory.Personal -> { /* Personal */ }
    TaskCategory.Other -> { /* Otros */ }
    // No hace falta 'else' porque el compilador sabe que son solo estos tres
}
// Si añadimos TaskCategory.Work y olvidamos actualizar el when:
// → Error de compilación: 'when' expression must be exhaustive
// → Nos salva de un bug en producción
```

|                         | enum class                    | sealed class                            |
|-------------------------|-------------------------------|-----------------------------------------|
| Instancias              | Únicas (singleton)            | Pueden tener constructores y estado     |
| Atributos por instancia | Solo los definidos en el enum | Cada subclase puede tener los suyos     |
| Atributos compartidos   | Solo constantes               | Sí, en la clase padre                   |
| Flexibilidad            | Limitada                      | Alta                                    |
| Uso típico              | Constantes con nombre         | Jerarquías de tipos con lógica variable |

### 📌 ¿Cuándo usar sealed class vs enum?

Usa enum cuando solo necesitas identificadores con nombre y un valor simple (colores, estados, días de la semana).

Usa sealed class cuando cada tipo puede tener sus propios atributos o comportamiento distinto.

Ejemplo: un resultado de red puede ser Success(data), Error(message), Loading → sealed class.

Los días de la semana → enum (todos son iguales estructuralmente).

En nuestra app, añadimos `isSelected` directamente en la sealed class para que todos los objetos hijos lo hereden sin repetir código:

```
// var isSelected = true → valor por defecto: categorías activas al inicio
sealed class TaskCategory(var isSelected: Boolean = true) {
    object Business : TaskCategory()
    object Personal : TaskCategory()
    object Other    : TaskCategory()
}
// Ahora CUALQUIER TaskCategory (Business, Personal u Other) tiene isSelected:
val cat: TaskCategory = TaskCategory.Business
cat.isSelected = false // ← funciona porque está definido en la sealed class padre
```

#### PASO 4 Crear TaskCategory.kt

Clic derecho sobre `com.dam.todoapp` > New > Kotlin Class/File > Class

Name: TaskCategory

Sustituye el contenido por el código de abajo.

```
package com.dam.todoapp

sealed class TaskCategory(var isSelected: Boolean = true) {
    object Business : TaskCategory()
    object Personal : TaskCategory()
    object Other    : TaskCategory()
}
```

## 6. Data class: Task

| Campo      | Tipo            | Descripción                                                  |
|------------|-----------------|--------------------------------------------------------------|
| name       | String          | Nombre/descripción de la tarea                               |
| category   | TaskCategory    | Categoría a la que pertenece (Business, Personal u Other)    |
| isSelected | Boolean = false | Si la tarea está marcada como completada. Por defecto: false |

#### PASO 5 Crear Task.kt

New > Kotlin Class/File > Class → Name: Task

```
package com.dam.todoapp

data class Task(
    val name: String,
    val category: TaskCategory,
    var isSelected: Boolean = false // var porque cambiará al marcar la tarea
)
// Uso:
// val t = Task("Comprar pan", TaskCategory.Personal)
// t.isSelected = true // se puede cambiar porque es var
// t.name = "otra" // ✗error: name es val
```

#### 📌 val vs var en data class

Usa `val` para campos que no deben cambiar (nombre, categoría). La tarea siempre pertenece a la misma categoría.

Usa var para campos que sí cambian (isSelected). Al marcar la tarea, necesitamos actualizar este campo. Esta distinción entre mutable e inmutable es una de las prácticas más importantes en Kotlin.

## 7. Proveedores de datos: companion object

En lugar de crear las listas directamente en la Activity, las extraemos a clases proveedoras. Este patrón lo usamos también en la SuperHeroApp: mantiene la Activity limpia y centraliza los datos de muestra en un único lugar.

### ¿Qué es un companion object?

Un companion object es un bloque dentro de una clase que actúa como si fuera estático (similar a static en Java). Sus miembros se llaman directamente sobre el nombre de la clase, sin necesidad de crear una instancia:

```
TaskProvider.getTasks() // ← no hace falta: TaskProvider().getTasks()
```

Es el equivalente Kotlin de los métodos de factoría estáticos de Java.

En nuestro caso lo usamos para centralizar los datos de prueba fuera de la Activity.

### PASO 6 Crear CategoryProvider.kt

New > Kotlin Class/File > Class → Name: CategoryProvider

```
package com.dam.todoapp

class CategoryProvider {
    companion object {
        fun getCategories(): List<TaskCategory> = listOf(
            TaskCategory.Business,
            TaskCategory.Personal,
            TaskCategory.Other
        )
    }
}

// Uso desde TodoActivity:
// private val categories = CategoryProvider.getCategories()
```

### PASO 7 Crear TaskProvider.kt

New > Kotlin Class/File > Class → Name: TaskProvider

```
package com.dam.todoapp

class TaskProvider {
    companion object {
        // MutableList porque la Activity añadirá y eliminará tareas en tiempo real
        fun getTasks(): MutableList<Task> = mutableListOf(
            Task("Tarea de negocios", TaskCategory.Business),
            Task("Tarea personal", TaskCategory.Personal),
            Task("Otra tarea", TaskCategory.Other)
        )
    }
}
```

```
// Uso desde TodoActivity:  
// private val tasks = TaskProvider.getTasks()
```

#### 📌 List vs MutableList

CategoryProvider devuelve List<TaskCategory> (inmutable): las categorías no se añaden ni eliminan, solo cambia su isSelected.

TaskProvider devuelve MutableList<Task> (mutable): el usuario añade tareas nuevas con el FAB, así que necesitamos poder llamar a tasks.add().

## PARTE II — INTERFAZ PRINCIPAL Y RECYCLERVIEW DE CATEGORÍAS

### 8. Crear TodoActivity

#### PASO 8 Crear TodoActivity

New > Activity > Empty Views Activity

Activity Name: TodoActivity

Layout Name: activity\_todo

Launcher Activity: ☒ MARCADO

Pulsa Finish.

#### 8.1 Layout activity\_todo.xml — FrameLayout raíz

##### ❓ ¿Por qué no un LinearLayout directamente?

Si el FAB estuviera dentro del LinearLayout, su posición vertical dependería de cuántas tareas haya en la lista. Con pocas tareas quedaría a mitad de pantalla; con muchas quedaría fuera de la vista.

El FrameLayout apila sus hijos y permite posicionar cualquier hijo en cualquier esquina mediante layout\_gravity, independientemente del contenido del otro hijo.

#### 8.2 Estilos XML — TodoSubtitle

```
<!-- res/values/themes.xml - añadir dentro de <resources> -->
<style name="TodoSubtitle">
    <item name="android:textAllCaps">true</item>
    <item name="android:textColor">@color/todo_subtitle_text</item>
    <item name="android:layout_marginHorizontal">32dp</item>
    <!-- Cada TextView que use este style hereda estos 3 atributos -->
</style>
<!-- Uso en el layout: -->
<TextView
    style="@style/TodoSubtitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/todo_categories" />
```

##### ❓ Ventaja de los estilos

Si mañana quieres cambiar el color de los subtítulos de toda la app, solo cambias el style en un lugar y todos los TextViews que lo usen se actualizan automáticamente.

Es el mismo principio que el de colors.xml: centralizar para facilitar el mantenimiento.

#### 8.3 activity\_todo.xml completo

#### PASO 9 Construir activity\_todo.xml

Abre activity\_todo.xml y sustituye el contenido con el XML de abajo.

Recuerda añadir el style TodoSubtitle en themes.xml antes.

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
```

```

android:layout_height="match_parent">
<!-- LinearLayout: contiene el contenido principal (título + RVs) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/todo_background_todo_app"
    android:orientation="vertical"
    tools:context=".TodoActivity">
    <!-- Título de bienvenida -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginHorizontal="32dp"
        android:layout_marginVertical="44dp"
        android:text="@string/todo_title"
        android:textAllCaps="true"
        android:textColor="@color/white"
        android:textSize="35sp"
        android:textStyle="bold" />
    <!-- Subtítulo Categorías (usa el style compartido) -->
    <TextView
        style="@style/TodoSubtitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/todo_categories" />
    <!-- RecyclerView HORIZONTAL para las categorías -->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvCategories"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <!-- Subtítulo Tareas -->
    <TextView
        style="@style/TodoSubtitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/todo_tasks" />
    <!-- RecyclerView VERTICAL para las tareas -->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvTasks"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>
<!-- FAB fijo en esquina inferior derecha -->
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fabAddTask"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="end|bottom"
    android:layout_margin="16dp"
    android:src="@drawable/ic_add"
    app:backgroundTint="@color/todo_accent" />
</FrameLayout>

```

✓ Estado actual:

- ✓ Layout de la pantalla principal con título, dos subtítulos, dos RecyclerViews y FAB.
- ✓ El FAB usa `android:layout_gravity="end|bottom"` y está fijo en la esquina.
- ✓ Los subtítulos usan el style `TodoSubtitle` (sin repetir atributos).
- Siguiente: crear el RecyclerView de categorías.

## 9. RecyclerView: funcionamiento interno

### 📌 Analogía con un restaurante

Imagina que en un restaurante hay 50 mesas pero solo 10 están ocupadas a la vez. En lugar de tener 50 juegos de cubiertos, el restaurante tiene 15 y los reutiliza: cuando una mesa se desocupa, los cubiertos se limpian y se llevan a la siguiente mesa que se ocupe.

El RecyclerView hace exactamente lo mismo con los widgets.

| Pieza        | Clase que crea el desarrollador                 | Responsabilidad                                                                                          |
|--------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| RecyclerView | En el XML del layout                            | El contenedor visual que gestiona el scroll y el reciclado                                               |
| Adapter      | CategoriesAdapter.kt /<br>TasksAdapter.kt       | Conecta la lista de datos con el RecyclerView. Sabe cuántos ítems hay y cómo crear/actualizar cada vista |
| ViewHolder   | CategoriesViewHolder.kt /<br>TasksViewHolder.kt | Guarda las referencias a los widgets de UNA celda y sabe cómo pintarla con datos concretos               |

```
// Vertical (por defecto): ítems uno debajo de otro
rvTasks.layoutManager = LinearLayoutManager(this)
// Horizontal: ítems uno al lado del otro (para las categorías)
rvCategories.layoutManager = LinearLayoutManager(
    this,
    LinearLayoutManager.HORIZONTAL,
    false // false = no invertir el orden
)
```

#### 📦 Dependencia RecyclerView

El RecyclerView es una librería de AndroidX. En proyectos recientes ya está incluida como dependencia transitiva. Si Android Studio da error al escribir RecyclerView en el XML, añade en build.gradle.kts:

```
implementation("androidx.recyclerview:recyclerview:1.3.2")
```

## 10. RecyclerView de categorías

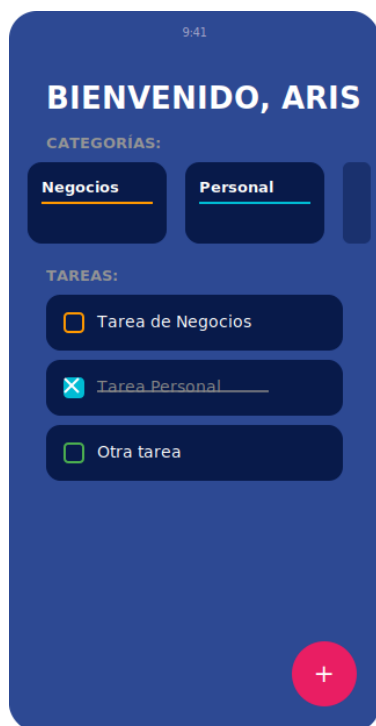


Fig. 3 – Las tres tarjetas de categoría (RecyclerView horizontal)

## 10.1 Layout item\_task\_category.xml

### PASO 8 Crear item\_task\_category.xml

Clic derecho sobre res/layout > New > Layout Resource File

File name: item\_task\_category

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/viewContainer"
    android:layout_width="160dp"
    android:layout_height="90dp"
    android:layout_marginHorizontal="8dp"
    android:layout_marginVertical="16dp"
    app:cardBackgroundColor="@color/todo_background_card"
    app:cardCornerRadius="16dp">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_margin="8dp"
        android:gravity="center_vertical"
        android:orientation="vertical">
        <!-- Nombre de la categoría -->
        <TextView
            android:id="@+id/tvCategoryName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textColor="@color/white"
            android:textSize="19sp"
            android:textStyle="bold"
            tools:text="Negocios" />
        <!-- Línea de color: su color se asigna desde Kotlin según la categoría -->
        <View
            android:id="@+id/divider"
            android:layout_width="match_parent"
            android:layout_height="2dp"
            android:layout_marginTop="4dp"
            tools:background="@color/white" />
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

## 10.2 CategoriesViewHolder.kt

### PASO 9 Crear CategoriesViewHolder.kt

New > Kotlin Class/File > Class → Name: CategoriesViewHolder

```
package com.dam.todoapp

import android.view.View
import android.widget.TextView
import androidx.cardview.widget.CardView
import androidx.core.content.ContextCompat
import androidx.recyclerview.widget.RecyclerView
import com.dam.todoapp.R

class CategoriesViewHolder(view: View) : RecyclerView.ViewHolder(view) {
    // Referencias a los widgets de la celda (se buscan una sola vez)
    private val tvCategoryName: TextView = view.findViewById(R.id.tvCategoryName)
    private val divider: View = view.findViewById(R.id.divider)
```

```

private val containerView: CardView = view.findViewById(R.id.viewContainer)

fun render(taskCategory: TaskCategory, onCategorySelected: (Int) -> Unit) {
    // 1. Color del fondo según si está seleccionada o deshabilitada
    val bgColor = if (taskCategory.isSelected) R.color.todo_background_card
        else R.color.todo_background_disabled
    containerView.setCardBackgroundColor(
        ContextCompat.getColor(viewContainer.context, bgColor)
    )
    // 2. Click listener: notifica la posición al Adapter
    itemView.setOnClickListener { onCategorySelected(layoutPosition) }
    // 3. Nombre y color de la línea según el tipo de categoría
    when (taskCategory) {
        TaskCategory.Business -> {
            tvCategoryName.text = "Negocios"
            divider.setBackgroundColor(
                ContextCompat.getColor(divider.context,
                    R.color.todo_business_category)
            )
        }
        TaskCategory.Personal -> {
            tvCategoryName.text = "Personal"
            divider.setBackgroundColor(
                ContextCompat.getColor(divider.context,
                    R.color.todo_personal_category)
            )
        }
        TaskCategory.Other -> {
            tvCategoryName.text = "Otros"
            divider.setBackgroundColor(
                ContextCompat.getColor(divider.context,
                    R.color.todo_other_category)
            )
        }
    }
}
}

```

#### ❗ viewContainer.context en lugar de this

Dentro del ViewHolder NO podemos usar 'this' para obtener el contexto porque no estamos en una Activity.

La solución es usar cualquier View como puente: view.context o containerView.context.

Cada View tiene una propiedad .context que devuelve el contexto de la Activity que la contiene.

Este es uno de los errores más comunes al escribir ViewHolders por primera vez.

## 10.3 CategoriesAdapter.kt

### PASO 12 Crear CategoriesAdapter.kt

New > Kotlin Class/File > Class → Name: CategoriesAdapter

```

package com.dam.todoapp

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.dam.todoapp.R

// El segundo parámetro es una función lambda: (Int) -> Unit
// Cuando el usuario pulse una celda, se llamará con su posición
class CategoriesAdapter(
    private val categories: List<TaskCategory>,
    private val onCategorySelected: (Int) -> Unit
) : RecyclerView.Adapter<CategoriesViewHolder>() {

```

```

        override fun getItemCount() = categories.size

        override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
CategoriesViewHolder {
            // Inflamos (creamos) la vista del ítem a partir del XML
            val view = LayoutInflater.from(parent.context)
                .inflate(R.layout.item_task_category, parent, false)
            return CategoriesViewHolder(view)
        }

        override fun onBindViewHolder(holder: CategoriesViewHolder, position: Int) {
            // Le decimos al ViewHolder qué datos pintar y le pasamos el lambda
            holder.render(categories[position], onCategorySelected)
        }
    }
}

```

## 11. Funciones lambda

---

```

// Tipo de una lambda: (TipoParametro) -> TipoRetorno
// (Int) -> Unit    =   función que recibe un Int y no devuelve nada
// () -> Boolean    =   función sin parámetros que devuelve Boolean
// (String, Int) -> String =   función con dos parámetros que devuelve String

// Declarar una variable de tipo lambda:
val saludo: (String) -> Unit = { nombre -> println("Hola, $nombre") }
saludo("Ana")    // → Hola, Ana

// Si la lambda tiene un solo parámetro, se puede usar 'it' en lugar de nombrarlo:
val saludo2: (String) -> Unit = { println("Hola, $it") }

// Pasarla como parámetro:
fun hacerAlgo(accion: (String) -> Unit) {
    accion("Mundo")
}
hacerAlgo { nombre -> println("Ejecutando con: $nombre") }

```

### 11.1 Trailing lambda

```

// Sin trailing lambda (válido pero menos legible):
CategoriesAdapter(categories, { position -> onCategorySelected(position) })
// Con trailing lambda (estilo Kotlin recomendado):
CategoriesAdapter(categories) { position -> onCategorySelected(position) }
// Equivalente con 'it' (cuando solo hay un parámetro):
CategoriesAdapter(categories) { onCategorySelected(it) }

```

## PARTE III — RECYCLERVIEW DE TAREAS Y LÓGICA DE LA ACTIVITY

### 12. Regla de oro del RecyclerView

```
// ✗MAL: modificar estado dentro del ViewHolder
itemView.setOnClickListener {
    task.isSelected = !task.isSelected    // parece que funciona...
    tvTask.paintFlags = ...                // ...pero tiene un grave bug
}
// El problema: cuando el RecyclerView hace scroll y recicla esta celda,
// la nueva tarea que ocupe este ViewHolder verá el estado visual del anterior.
// Resultado: tareas que aparecen marcadas sin haber sido pulsadas.

// ✔BIEN: notificar al Adapter para que repinte la celda con los datos correctos
itemView.setOnClickListener { onCategorySelected(layoutPosition) }
// En la Activity: tasks[position].isSelected = !tasks[position].isSelected
// Luego: tasksAdapter.notifyDataSetChanged()
```

#### 📌 Regla de oro del RecyclerView

El ViewHolder NUNCA debe modificar directamente el estado de los datos.

El ViewHolder solo PINTA lo que le dice el objeto de datos.

Cuando el usuario interactúa, se notifica al Adapter con la posición, la Activity actualiza los datos y luego notifica al Adapter para que repinte.

Este flujo unidireccional (datos → vista) evita los bugs más comunes de los RecyclerViews.

### 13. CheckBox y tachado de texto

| Atributo           | Descripción                                                            |
|--------------------|------------------------------------------------------------------------|
| android:checked    | Estado inicial: true o false                                           |
| android:buttonTint | Color del checkbox (recuadro y marca). Acepta @color/ o ColorStateList |
| cbTask.isChecked   | Propiedad Kotlin para leer o cambiar el estado desde código            |

```
// Tachar el texto:
tvTask.paintFlags = tvTask.paintFlags or Paint.STRIKE_THRU_TEXT_FLAG
// Equivale a: paintFlags = paintFlags | STRIKE_THRU (OR a nivel de bits)

// Eliminar el tachado:
tvTask.paintFlags = tvTask.paintFlags and Paint.STRIKE_THRU_TEXT_FLAG.inv()
// .inv() → inversión bit a bit (NOT). AND con el NOT de la flag = quitar esa flag

// IMPORTANTE: asignar el texto DESPUÉS de los paintFlags
tvTask.text = task.name    // ← siempre al final
```

```
import android.content.res.ColorStateList
import androidx.core.content.ContextCompat

// Obtener el color real (Int) a partir de R.color
val colorInt = ContextCompat.getColor(cbTask.context, R.color.todo_business_category)
// Crear el ColorStateList con un único color
cbTask.buttonTintList = ColorStateList.valueOf(colorInt)
```

### 14. RecyclerView de tareas

## 14.1 Layout item\_todo\_task.xml

### PASO 13 Crear item\_todo\_task.xml

New > Layout Resource File → Name: item\_todo\_task

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginVertical="4dp"
    app:cardBackgroundColor="@color/todo_background_card"
    app:cardCornerRadius="16dp">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:paddingVertical="8dp">
        <!-- CheckBox: muestra si la tarea está completada -->
        <CheckBox
            android:id="@+id/cbTask"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />
        <!-- Nombre de la tarea -->
        <TextView
            android:id="@+id/tvTask"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textColor="@color/white"
            android:textSize="18sp"
            tools:text="Tarea de prueba" />
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

## 14.2 TasksViewHolder.kt

### PASO 14 Crear TasksViewHolder.kt

```
package com.dam.todoapp

import android.content.res.ColorStateList
import android.graphics.Paint
import android.view.View
import android.widget.CheckBox
import android.widget.TextView
import androidx.core.content.ContextCompat
import androidx.recyclerview.widget.RecyclerView
import com.dam.todoapp.R

class TasksViewHolder(view: View) : RecyclerView.ViewHolder(view) {
    private val tvTask: TextView = view.findViewById(R.id.tvTask)
    private val cbTask: CheckBox = view.findViewById(R.id.cbTask)

    fun render(task: Task) {
        // 1. Tachar o no el texto según isSelected
        if (task.isSelected) {
            tvTask.paintFlags = tvTask.paintFlags or Paint.STRIKE_THRU_TEXT_FLAG
        } else {
            tvTask.paintFlags = tvTask.paintFlags and
            Paint.STRIKE_THRU_TEXT_FLAG.inv()
        }
    }
}
```

```

    }
    // 2. Nombre de la tarea (siempre DESPUÉS de los paintFlags)
    tvTask.text = task.name
    // 3. Estado del checkbox
    cbTask.isChecked = task.isSelected
    // 4. Color del checkbox según la categoría de la tarea
    val colorRef = when (task.category) {
        TaskCategory.Business -> R.color.todo_business_category
        TaskCategory.Personal -> R.color.todo_personal_category
        TaskCategory.Other -> R.color.todo_other_category
    }
    cbTask.buttonTintList = ColorStateList.valueOf(
        ContextCompat.getColor(cbTask.context, colorRef)
    )
}
}

```

## 14.3 TasksAdapter.kt

### PASO 15 Crear TasksAdapter.kt

```

package com.dam.todoapp

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.dam.todoapp.R

class TasksAdapter(
    var tasks: List<Task>, // var: se reemplaza al filtrar
    private val onTaskSelected: (Int) -> Unit
) : RecyclerView.Adapter<TasksViewHolder>() {

    override fun getItemCount() = tasks.size

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): TasksViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_todo_task, parent, false)
        return TasksViewHolder(view)
    }

    override fun onBindViewHolder(holder: TasksViewHolder, position: Int) {
        holder.render(tasks[position])
        // El click listener se pone aquí (no en el ViewHolder) para evitar bugs
        holder.itemView.setOnClickListener { onTaskSelected(position) }
    }
}

```

#### 🔗 ¿Por qué el click listener está en onBindViewHolder y no en render()?

En CategoriesAdapter pasamos la lambda al ViewHolder y la asignamos dentro de render(). Aquí hacemos lo contrario: el listener se pone directamente en onBindViewHolder, fuera del ViewHolder.

El motivo es evitar un bug sutil con `layoutPosition`. Si pusiéramos el listener dentro de `render()`:

```
itemView.setOnClickListener { onCategorySelected(layoutPosition) }
```

`layoutPosition` devuelve la posición visual del ViewHolder en el momento del click. En una lista que se filtra dinámicamente (al pulsar una categoría, la lista de tareas se recalcula por completo), esa posición puede haberse desfasado respecto al índice real del elemento en `tasks[]`.

Al asignar el listener en `onBindViewHolder`, la posición que captura el closure es la del momento exacto del bind, que siempre corresponde a `tasks[position]` correcto.

Ambas formas funcionan en la mayoría de los casos. La versión con el listener en `onBindViewHolder` es la que recomiendan las guías oficiales de Google para listas con filtrado o animaciones.

## PARTE IV — LÓGICA DE TODOACTIVITY Y DIALOG

### 15. TodoActivity.kt — esqueleto y lógica completa

#### PASO 16 Escribir el esqueleto de TodoActivity.kt

```
package com.dam.todoapp

import android.app.Dialog
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.RadioButton
import android.widget.RadioGroup
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.google.android.material.floatingactionbutton.FloatingActionButton
import com.dam.todoapp.TaskCategory.*

class TodoActivity : AppCompatActivity() {
    // — Datos (cargados desde los providers) —————
    private val categories = CategoryProvider.getCategories()
    private val tasks = TaskProvider.getTasks()
    // — Referencias a los widgets —————
    private lateinit var rvCategories: RecyclerView
    private lateinit var categoriesAdapter: CategoriesAdapter
    private lateinit var rvTasks: RecyclerView
    private lateinit var tasksAdapter: TasksAdapter
    private lateinit var fabAddTask: FloatingActionButton

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_todo)
        initComponents()
        initUI()
        initListeners()
    }

    private fun initComponents() {
        rvCategories = findViewById(R.id.rvCategories)
        rvTasks = findViewById(R.id.rvTasks)
        fabAddTask = findViewById(R.id.fabAddTask)
    }

    private fun initListeners() {
        fabAddTask.setOnClickListener { showDialog() }
    }

    private fun initUI() {
        categoriesAdapter = CategoriesAdapter(categories) { position ->
            onCategorySelected(position)
        }
        rvCategories.layoutManager = LinearLayoutManager(
            this, LinearLayoutManager.HORIZONTAL, false)
        rvCategories.adapter = categoriesAdapter

        tasksAdapter = TasksAdapter(tasks) { position ->
            onTaskSelected(position)
        }
        rvTasks.layoutManager = LinearLayoutManager(this)
        rvTasks.adapter = tasksAdapter
    }

    private fun onTaskSelected(position: Int) { /* TODO */ }
    private fun onCategorySelected(position: Int) { /* TODO */ }
}
```

### 15.1 Repintado básico: notifyDataSetChanged y notifyItemChanged

Antes de añadir el filtrado por categoría, conectamos los clicks para que la app ya repinte los RecyclerViews correctamente. Aquí vemos la diferencia entre los dos métodos de notificación.

| Método                 | Qué repinta                   | Cuándo usarlo                                                 |
|------------------------|-------------------------------|---------------------------------------------------------------|
| notifyDataSetChanged() | TODA la lista de golpe        | Cuando pueden cambiar muchos ítems a la vez (filtrado, orden) |
| notifyItemChanged(pos) | Solo la celda en esa posición | Cuando sabemos exactamente qué celda ha cambiado              |

#### PASO 17 Implementar onTaskSelected() — marcar tarea

Cuando el usuario pulsa una tarea, invertimos su estado y pedimos al adapter que repinte toda la lista de tareas:

```
private fun onTaskSelected(position: Int) {  
    // 1. Invertir el estado de la tarea pulsada  
    tasks[position].isSelected = !tasks[position].isSelected  
    // 2. Pedir al adapter que repinte TODA la lista  
    // (usamos notifyDataSetChanged porque cualquier celda podría cambiar  
    // visualmente)  
    tasksAdapter.notifyDataSetChanged()  
}
```

#### PASO 18 Implementar onCategorySelected() — activar/desactivar categoría

Cuando el usuario pulsa una categoría, solo cambia ESA celda, así que usamos notifyItemChanged en lugar de repintar todo:

```
private fun onCategorySelected(position: Int) {  
    // 1. Invertir el estado de la categoría pulsada  
    categories[position].isSelected = !categories[position].isSelected  
    // 2. Repintar SOLO esa celda (más eficiente: sabemos exactamente cuál cambió)  
    categoriesAdapter.notifyItemChanged(position)  
    // 3. También repintar las tareas (TODO: añadir filtro en el siguiente paso)  
    tasksAdapter.notifyDataSetChanged()  
}
```

#### 📌 ¿Por qué notifyItemChanged para categorías y notifyDataSetChanged para tareas?

Al pulsar una categoría, sabemos exactamente qué celda cambia: la que se ha pulsado. Por eso notifyItemChanged(position) es suficiente y más eficiente.

Al pulsar una tarea, solo cambia esa celda visualmente (tachado + checkbox). No cambia el filtro, así que notifyDataSetChanged() es suficiente — no hace falta recalcular nada.

Regla práctica: si sabes qué celda cambia → notifyItemChanged. Si puede cambiar la lista entera → notifyDataSetChanged. Si cambia el filtro → updateTasks().

✓ Estado intermedio — la app ya repinta:

✓ Pulsar una tarea la marca/desmarca con tachado y checkbox.

✓ Pulsar una categoría cambia su color (activa ↔ deshabilitada).

✗ Las tareas aún no se filtran al desactivar una categoría. Lo añadimos a continuación.

► Siguiente: añadir la lógica de filtrado.

## 15.2 Filtrado por categoría: filter()

Ahora que los RecyclerViews repintan correctamente, añadimos la lógica que oculta las tareas cuya categoría está desactivada. Para eso usamos la función `filter()` de las colecciones Kotlin.

```
// filter {} devuelve una NUEVA lista con solo los elementos que cumplen la condición
val numeros = listOf(1, 2, 3, 4, 5, 6)
val pares = numeros.filter { it % 2 == 0 } // → [2, 4, 6]

// En nuestra app lo aplicamos en dos pasos:
// Paso 1 – qué categorías están activas ahora mismo:
val selectedCategories = categories.filter { it.isSelected }
// Paso 2 – solo las tareas cuya categoría está en esa lista:
val filteredTasks = tasks.filter { selectedCategories.contains(it.category) }
```

### PASO 19 Implementar `updateTasks()` y conectarla a `onCategorySelected()`

Creamos `updateTasks()` con la lógica del filtro y la llamamos desde `onCategorySelected()`, que es la única función que cambia qué categorías están activas. `onTaskSelected()` no la necesita: solo cambia el estado visual de una tarea, no el filtro.

```
private fun updateTasks() {
    // 1. Categorías que siguen activas (isSelected = true)
    val selectedCategories: List<TaskCategory> = categories.filter { it.isSelected }
    // 2. Solo las tareas cuya categoría está activa
    val filteredTasks = tasks.filter { selectedCategories.contains(it.category) }
    // 3. Reemplazar la lista del adapter y repintar
    tasksAdapter.tasks = filteredTasks
    tasksAdapter.notifyDataSetChanged()
}

// onTaskSelected NO llama a updateTasks(): solo repinta la tarea tocada
private fun onTaskSelected(position: Int) {
    tasks[position].isSelected = !tasks[position].isSelected
    tasksAdapter.notifyDataSetChanged()
}

// onCategorySelected SÍ llama a updateTasks(): el filtro ha cambiado
private fun onCategorySelected(position: Int) {
    categories[position].isSelected = !categories[position].isSelected
    categoriesAdapter.notifyItemChanged(position)
    updateTasks() // ← recalcula qué tareas son visibles
}
```

#### ❓ ¿Por qué `tasksAdapter.tasks = filteredTasks` y no modificar la lista original?

`tasks[]` es la lista COMPLETA de todas las tareas (nunca la tocamos en el filtrado).

Lo que cambia en el adapter es la vista: le pasamos una sublista de las tareas visibles.

Si el usuario reactiva una categoría, `updateTasks()` recalcula la sublista y aparecen de nuevo las tareas que estaban ocultas. Si hubiéramos borrado de `tasks[]`, las habríamos perdido para siempre.

✓ La pantalla principal está completamente funcional:

✓ RecyclerView horizontal con las 3 categorías con colores.

- ✓ RecyclerView vertical con las tareas.
- ✓ Pulsar una categoría la deshabilita y oculta sus tareas inmediatamente.
- ✓ Reactivar una categoría restaura sus tareas.
- ✓ Pulsar una tarea la marca/desmarca con tachado y checkbox.
- Siguiente: crear el diálogo para añadir tareas.

## 16. Dialog para añadir tareas

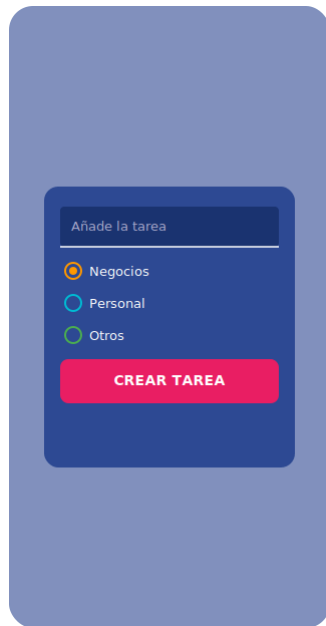


Fig. 4 – Diálogo para añadir nueva tarea

| Tipo de Dialog    | Cuándo usarlo                                                   |
|-------------------|-----------------------------------------------------------------|
| Dialog (básico)   | Cuando necesitas una vista completamente personalizada          |
| AlertDialog       | Para mensajes de confirmación con botones Aceptar/Cancelar      |
| BottomSheetDialog | Para paneles que suben desde la parte inferior (estilo moderno) |
| DatePickerDialog  | Para seleccionar fechas                                         |
| TimePickerDialog  | Para seleccionar horas                                          |

### 📌 Dialog vs nueva Activity

Un Dialog es adecuado cuando la acción es breve y puntual (añadir una tarea, confirmar un borrado).

Una Activity nueva es mejor cuando la acción es compleja y requiere su propio espacio (formulario largo, configuración, pantalla de detalles).

Regla: si el usuario puede completar la acción en menos de 30 segundos y con pocos campos, usa Dialog.

### 16.1 RadioGroup y RadioButton

| Atributo/Propiedad     | Descripción                                |
|------------------------|--------------------------------------------|
| android:checked="true" | El RadioButton arranca marcado por defecto |
| android:buttonTint     | Color del círculo del RadioButton          |
| android:text           | Texto junto al RadioButton                 |

|                                                |                                                             |
|------------------------------------------------|-------------------------------------------------------------|
| <code>rgCategories.checkedRadioButtonId</code> | ID del RadioButton actualmente marcado (en Kotlin)          |
| <code>rgCategories.findViewById(id)</code>     | Obtener el RadioButton marcado del grupo para leer su texto |

## 16.2 dialog\_task.xml

### PASO 20 Crear dialog\_task.xml

New > Layout Resource File → Name: dialog\_task

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="250dp"
    android:layout_height="wrap_content"
    android:background="@color/todo_background_todo_app"
    android:orientation="vertical"
    android:padding="16dp">
    <EditText
        android:id="@+id/etTask"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:backgroundTint="@color/white"
        android:hint="@string/todo_dialog_add_task"
        android:textColor="@color/white"
        android:textColorHint="@color/white" />
    <RadioGroup
        android:id="@+id/rgCategories"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
        <RadioButton
            android:id="@+id/rbBusiness"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:buttonTint="@color/todo_business_category"
            android:checked="true"
            android:text="@string/todo_dialog_category_business"
            android:textColor="@color/white" />
        <RadioButton
            android:id="@+id/rbPersonal"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:buttonTint="@color/todo_personal_category"
            android:text="@string/todo_dialog_category_personal"
            android:textColor="@color/white" />
        <RadioButton
            android:id="@+id/rbOther"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:buttonTint="@color/todo_other_category"
            android:text="@string/todo_dialog_category_other"
            android:textColor="@color/white" />
    </RadioGroup>
    <Button
        android:id="@+id/btnAddTask"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:backgroundTint="@color/todo_accent"
        android:text="@string/todo_dialog_add_task_button" />
</LinearLayout>
```

## 16.3 Usar Dialog en Kotlin

```
val dialog = Dialog(this)
dialog setContentView(R.layout.dialog_task)
```

```

dialog.show() // mostrar (SIEMPRE al final)

// Para buscar widgets dentro del Dialog (NO en la Activity):
val btnAddTask: Button = dialog.findViewById(R.id.btnAddTask)
// IMPORTANTE: usar dialog.findViewById(), NO solo findViewById()

```

```

val selectedId = rgCategories.checkedRadioButtonId
val selectedRadioButton: RadioButton = rgCategories.findViewById(selectedId)
val currentCategory: TaskCategory = when (selectedRadioButton.text) {
    getString(R.string.todo_dialog_category_business) -> TaskCategory.Business
    getString(R.string.todo_dialog_category_personal) -> TaskCategory.Personal
    else -> TaskCategory.Other
}

```

## 16.4 showDialog() completo

### PASO 21 Implementar showDialog() en TodoActivity.kt

```

private fun showDialog() {
    val dialog = Dialog(this)
    dialog setContentView(R.layout.dialog_task)

    val btnAddTask: Button = dialog.findViewById(R.id.btnAddTask)
    val etTask: EditText = dialog.findViewById(R.id.etTask)
    val rgCategories: RadioGroup = dialog.findViewById(R.id.rgCategories)

    btnAddTask.setOnClickListener {
        val taskName = etTask.text.toString()
        if (taskName.isNotEmpty()) {
            val selectedId = rgCategories.checkedRadioButtonId
            val selectedRB: RadioButton = rgCategories.findViewById(selectedId)
            val category: TaskCategory = when (selectedRB.text) {
                getString(R.string.todo_dialog_category_business) -> Business
                getString(R.string.todo_dialog_category_personal) -> Personal
                else -> Other
            }
            tasks.add(Task(taskName, category))
            updateTasks()
            dialog.hide()
        }
    }
    dialog.show() // mostrar el dialog SIEMPRE al final
}

```

- ✓ La app ToDoApp está completamente terminada:
- ✓ RecyclerView horizontal de categorías con habilitar/deshabilitar.
- ✓ RecyclerView vertical de tareas con marcado y tachado.
- ✓ Filtrado de tareas por categoría activa.
- ✓ FAB fijo en esquina inferior derecha.
- ✓ Dialog para añadir nuevas tareas con RadioGroup de categorías.
- ✓ Sealed class TaskCategory garantizando tipos seguros.
- ✓ Funciones lambda para comunicar clicks entre ViewHolder, Adapter y Activity.

## RESUMEN DE LA UNIDAD 4

### Código completo — referencia rápida

| Fichero                 | Función / Responsabilidad                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------|
| TaskCategory.kt         | Sealed class con 3 objetos (Business, Personal, Other) y atributo isSelected compartido        |
| Task.kt                 | Data class con name (val), category (val), isSelected (var = false)                            |
| activity_todo.xml       | FrameLayout > [LinearLayout con título + 2 RVs] + [FAB con layout_gravity=end bottom]          |
| TodoActivity.kt         | initComponent/UI/Listeners, onTaskSelected, onCategorySelected, updateTasks, showDialog        |
| item_task_category.xml  | CardView (160×90dp) con TextView nombre + View divisor de 2dp                                  |
| CategoriesViewHolder.kt | render(): color fondo (selected/disabled), click listener, texto + color divisor por categoría |
| CategoriesAdapter.kt    | getItemCount, onCreateViewHolder, onBindViewHolder con lambda (Int)->Unit                      |
| item_todo_task.xml      | CardView con LinearLayout horizontal: CheckBox + TextView                                      |
| TasksViewHolder.kt      | render(): paintFlags tachado, texto, isChecked, buttonTintList con ColorStateList              |
| TasksAdapter.kt         | var tasks (mutable), click en onBindViewHolder con lambda (Int)->Unit                          |
| dialog_task.xml         | LinearLayout (250dp) con EditText, RadioGroup (3 RadioButtons) y Button                        |

### Conceptos clave

| Concepto                  | Descripción                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Sealed class              | Clase con subclases limitadas al mismo fichero. El compilador garantiza exhaustividad en when.                                    |
| Sealed class con atributo | var isSelected en la sealed class padre: todos los objetos hijos lo heredan.                                                      |
| Data class Task           | Modelo de datos con val/var según si el campo cambia. Genera equals/toString/copy automáticamente.                                |
| RecyclerView              | Lista eficiente que recicla las vistas. No crea un widget por elemento, sino que reutiliza los visibles.                          |
| Adapter                   | Intermediario entre lista de datos y RecyclerView. Implementa getItemCount, onCreateViewHolder, onBindViewHolder.                 |
| ViewHolder                | Guarda referencias a los widgets de UNA celda. La función render() pinta los datos.                                               |
| LayoutManager             | Define la disposición: LinearLayoutManager.HORIZONTAL o VERTICAL.                                                                 |
| Lambda (Int) -> Unit      | Función anónima que se pasa como parámetro. Permite comunicar clicks de ViewHolder → Activity.                                    |
| Trailing lambda           | Lambda como último parámetro: se puede sacar fuera de los paréntesis. Estilo idiomático Kotlin.                                   |
| Regla de oro RV           | El ViewHolder solo pinta; nunca modifica el estado. El click notifica posición → Activity actualiza datos → notifyDataSetChanged. |
| notifyItemChanged(pos)    | Repinta solo una celda (más eficiente que notifyDataSetChanged para cambios puntuales).                                           |
| .filter { condición }     | Función de listas que devuelve una nueva lista con los elementos que cumplen la condición.                                        |
| Dialog                    | Ventana flotante sobre la Activity. setContentView() infla el layout; dialog.findViewById() para los widgets.                     |
| RadioGroup + RadioButton  | Selección exclusiva: solo un RadioButton puede estar marcado a la vez dentro de un grupo.                                         |

|                               |                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------|
| Paint.STRIKE_THRU_TEXT_FLAG   | Flag de pintura para tachar el texto de un TextView.                                     |
| ColorStateList.valueOf(color) | Convierte un Int de color en el ColorStateList que necesita buttonTintList del CheckBox. |
| android:layout_gravity        | En FrameLayout: posiciona el hijo en una esquina. end bottom = esquina inferior derecha. |
| Style XML                     | Agrupar atributos repetidos en un @style que se aplica con style="@style/nombre".        |

## EJERCICIOS

### 📖 Instrucciones

Todos los ejercicios parten del proyecto ToDoApp terminado.

Ejecuta la app en el emulador tras cada cambio para verificar el resultado.

### 📖 Ejercicio 1 – Borrar tarea con click largo

Añade la posibilidad de borrar una tarea manteniendo pulsada (long click):

- En TasksAdapter.onBindViewHolder, añade un longClickListener:  
holder.itemView.setOnLongClickListener { onTaskLongClick(position); true }
- Añade el segundo parámetro lambda al adapter: onTaskLongClick: (Int) -> Unit
- En TodoActivity, implementa la función que elimina la tarea:  
tasks.removeAt(position)  
updateTasks()
- Pasa la nueva lambda al crear el TasksAdapter en initUI().

Nota: setOnLongClickListener devuelve Boolean (true = el evento fue consumido).

### 📖 Ejercicio 2 – Contador de tareas pendientes

Muestra en la pantalla principal el número de tareas pendientes (isSelected = false).

- Añade un TextView con id tvPendingCount debajo del título en activity\_todo.xml.
- Decláralo como lateinit var en TodoActivity y asígnalo en initComponents().
- En updateTasks(), calcula las pendientes y actualiza el TextView:  
val pending = tasks.count { !it.isSelected }  
tvPendingCount.text = "\$pending tareas pendientes"
- Llama también a esta actualización en onTaskSelected().

### 📖 Ejercicio 3 – Validación mejorada en el Dialog

Actualmente el Dialog no hace nada si el campo está vacío (isEmpty lo valida).

Mejora la experiencia de usuario:

- Deshabilita el botón 'Crear tarea' al abrir el dialog: btnAddTask.isEnabled = false
- Añade un TextWatcher al EditText que habilite/deshabilite el botón:  
etTask.addTextChangedListener { btnAddTask.isEnabled = it.toString().isEmpty() }
- Verifica que el botón se activa al escribir y se desactiva al borrar todo el texto.

### 📖 Ejercicio 4 – Cuarta categoría 'Urgente'

Añade una categoría 'Urgente' a la app:

- a) Añade object Urgent : TaskCategory() en TaskCategory.kt.
  - b) Android Studio marcará en rojo todos los when que no cubran Urgent → corrígelos.
  - c) Añade el color en colors.xml: todo\_urgent\_category (#F44336 – rojo).
  - d) Añade en strings.xml: todo\_dialog\_category\_urgent con el texto 'Urgente'.
  - e) Añade el RadioButton correspondiente en dialog\_task.xml.
  - f) Trata el nuevo caso en el when de showDialog().
- Observa cómo la sealed class te obliga a actualizar todos los when.

#### 📌 Ejercicio 5 – (Reto) Persistencia básica con SharedPreferences

Actualmente las tareas se pierden al cerrar la app. Implementa persistencia básica:

- a) Guarda el número de tareas al salir: override fun onDestroy() { ... }
- b) Usa SharedPreferences para guardar un JSON simple de las tareas:  
val prefs = getSharedPreferences("todo\_prefs", Context.MODE\_PRIVATE)  
prefs.edit().putInt("task\_count", tasks.size).apply()
- c) En onCreate(), carga el valor guardado y muéstralo en un Toast.

Nota: la persistencia completa con JSON requiere la librería Gson (build.gradle.kts).

Este ejercicio es introductorio: en unidades posteriores veremos Room Database.

## APÉNDICE — Organización del proyecto en paquetes

Una vez que la app funciona, es buen momento para ordenar el código en paquetes. Android Studio permite reorganizar ficheros con Refactor sin romper las importaciones.

### ¿Por qué organizar en paquetes?

En este momento todos los ficheros (TaskCategory.kt, Task.kt, CategoriesAdapter.kt, etc.) conviven en el paquete raíz com.dam.todoapp. Con 11 ficheros ya empieza a ser difícil encontrar lo que buscas. La convención en Android es agrupar por responsabilidad:

| Paquete  | Contiene                                                                           | ¿Por qué aquí?                                                                                                      |
|----------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| model/   | Task.kt, TaskCategory.kt, TaskProvider.kt, CategoryProvider.kt                     | Datos puros: data class, sealed class y proveedores de datos iniciales. No dependen de ninguna vista ni de Android. |
| adapter/ | CategoriesAdapter.kt, CategoriesViewHolder.kt, TasksAdapter.kt, TasksViewHolder.kt | Todo el código de RecyclerView va junto. Si necesitas cambiar cómo se pinta una celda, sabes dónde buscar.          |
| (raíz)   | TodoActivity.kt, MainActivity.kt                                                   | Las Activities quedan en la raíz. Son el punto de entrada de la pantalla.                                           |

### Estructura final del proyecto

```
com.dam.todoapp/  
├── model/  
│   ├── Task.kt  
│   ├── TaskCategory.kt  
│   ├── TaskProvider.kt  
│   └── CategoryProvider.kt  
├── adapter/  
│   ├── CategoriesAdapter.kt  
│   ├── CategoriesViewHolder.kt  
│   ├── TasksAdapter.kt  
│   └── TasksViewHolder.kt  
└── TodoActivity.kt
```

### Cómo mover ficheros con Android Studio Refactor

No es necesario cortar y pegar manualmente. Android Studio actualiza todas las importaciones de forma automática si usas Refactor > Move:

- **Crear el paquete model/:** Clic derecho sobre com.dam.todoapp > New > Package. Escribe model (Android Studio crea la carpeta).
- **Mover Task.kt:** Clic derecho sobre Task.kt > Refactor > Move. Selecciona com.dam.todoapp.model. Acepta actualizar las importaciones.
- **Mover TaskCategory.kt:** Repite el proceso con TaskCategory.kt al paquete model/.
- **Crear el paquete adapter/:** Igual que el paso 1, pero con el nombre adapter.
- **Mover los 4 ficheros de adapter:** Mueve CategoriesAdapter, CategoriesViewHolder, TasksAdapter y TasksViewHolder al paquete adapter/. Puedes seleccionarlos todos a la vez con Ctrl/Cmd + clic y hacer Refactor > Move una sola vez.

- **Verificar compilación:** Pulsa Build > Make Project (Ctrl+F9). Si compila sin errores, todo está correcto. Si aparece algún import en rojo, Android Studio lo señala: Alt+Enter para añadirlo automáticamente.

#### 📌 ¿Cuándo hacer la reorganización?

Lo más práctico es terminar primero la app (que funcione al 100 %) y luego reorganizar. Así puedes verificar que nada se rompe con un solo Build.

Si creas el paquete desde el principio, cada fichero nuevo debe crearse ya en su paquete correcto: clic derecho sobre model/ o adapter/ en lugar del paquete raíz.

## Importaciones que cambian

Tras mover los ficheros, las importaciones se actualizan así:

```
// ANTES (todo en la raíz):
import com.dam.todoapp.Task
import com.dam.todoapp.TaskCategory
import com.dam.todoapp.CategoriesAdapter

// DESPUÉS (con paquetes):
import com.dam.todoapp.model.Task
import com.dam.todoapp.model.TaskCategory
import com.dam.todoapp.adapter.CategoriesAdapter

// Android Studio las actualiza automáticamente con Refactor > Move.
// Solo tienes que confirmar el diálogo "Update usages? → Yes".
```

#### 📌 ¿Por qué no creamos una capa data/ o repository/?

En apps más grandes se añade una capa data/ (repositorios, fuentes de datos, base de datos).

En este proyecto los datos están en listas en memoria, así que model/ es suficiente.

Cuando veamos Room Database en la siguiente unidad, ese será el momento de añadir data/.

## Comparativa antes / después

| Sin paquetes (antes)                               | Con paquetes (después)                |
|----------------------------------------------------|---------------------------------------|
| 11 ficheros en la raíz                             | 2 en model/, 4 en adapter/, 1 en raíz |
| Difícil saber si un fichero es "datos" o "vista"   | La carpeta lo dice inmediatamente     |
| Al buscar "Adapter" hay que leer todos los nombres | Abres adapter/ y están todos          |
| Copiar el código a otro proyecto es tedioso        | Copias la carpeta model/ completa     |

## Recursos adicionales

#### 📌 Para saber más

RecyclerView oficial: <https://developer.android.com/develop/ui/views/layout/recyclerview>

Sealed classes en Kotlin: <https://kotlinlang.org/docs/sealed-classes.html>

Dialog en Android: <https://developer.android.com/develop/ui/views/components/dialogs>

Lambdas en Kotlin: <https://kotlinlang.org/docs/lambdas.html>

RadioButton y RadioGroup: <https://developer.android.com/develop/ui/views/components/radiobutton>

Guía de arquitectura Android: <https://developer.android.com/topic/architecture>